上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

人工智能实践教学中心
Artificial Intelligence Practice Teaching Center

计算机工程实践

章鸣铭
brucechang@sjtu.edu.cn

饮水思源 · 爱国荣校



宇树Go2



Chapter 3

传感器与感知

核心内容：

本课件主要分为三大部分内容：第一部分讲解机器狗基础传感与状态读取，包括电池电量、机身姿态角度、舵机角度的获取代码与实现方法；第二部分介绍机器狗教育扩展库的实用功能，涵盖屏幕绘图、视频录制等多媒体相关操作；第三部分学习 OpenCV 图像处理基础，掌握形状检测、图像裁剪、灰度转换、图片旋转等常用视觉功能。整体以代码实操为主，由硬件读取到功能应用，再到视觉基础，层层递进，帮助掌握机器狗控制与视觉处理的核心技能。

01

读取机器狗信息

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

读取表示机器狗状态的相关函数

读取机器狗电池电量

read_battery ()

- 读取当前电池电量， 读取成功则返回1-100的整数，代表电池剩余电量百分比， 读取失败则返回0。
- 在远程链接之后，机器狗在运动的时候查看电量不方便，**我们会用在每次启动脚本的时候使用这个方法查询电量，可以通过查询电量来决定是否进行本轮次操作。**
- 如右图代码，通过xgolib库中的XGO类连接 XGO Lite 机器狗，封装一个传感器读取类，**专门用于获取并打印机器狗的电池电量。这样就提供了安全的电池状态监测框架**

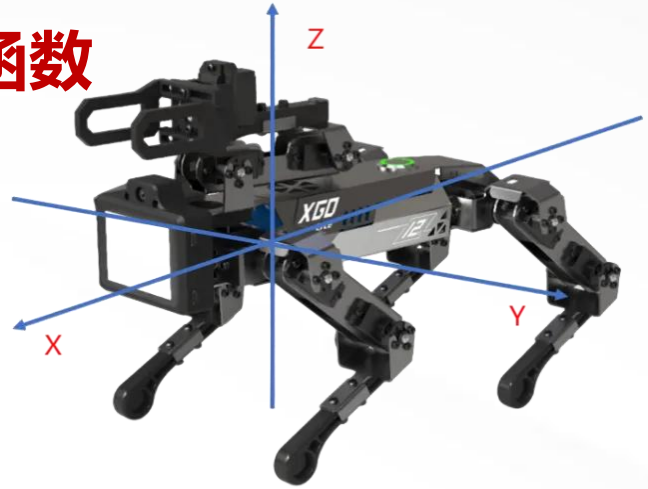
```python
# 导入运动库
from xgolib import XGO

class RobotDogSensors:
    def __init__(self):
        self.dog = XGO("xgolite")

    def read_battery_level(self):
        """
        读取并返回机器狗的电量百分比。
        """
        battery_level = self.dog.read_battery()
        return battery_level

# 创建传感器对象
sensors = RobotDogSensors()
# 读取电量
battery_level = sensors.read_battery_level()
print(f"Current battery level: {battery_level}%")
```

读取表示机器狗状态的相关函数



读取机器狗姿态角度

读取姿态角度有三个方法：

read_roll()　## 滚转角（绕X轴旋转，代表机器狗机身侧倾的角度）

read_pitch()　## 俯仰角（绕Y轴旋转，俯仰角代表机器狗机身前后倾角度）

read_yaw()　## 偏航角（绕Z轴旋转，偏航角代表机身左右转动的角度）

- ✓ 读取当前姿态角度，读取成功则返回浮点数，单位为度，读取失败则返回0。
- ✓ 在这些方法中，**最重要的是 read_yaw()** 获取的偏航角信息。可以用来准确控制机身角度。

```python
from xgolib import XGO

class RobotDogSensors:
    def __init__(self):
        self.dog = XGO("xgolite")

    def read_pose(self):
        """
        读取并返回机器狗的位姿角度。
        """
        roll = self.dog.read_roll()
        pitch = self.dog.read_pitch()
        yaw = self.dog.read_yaw()
        return roll, pitch, yaw

# 创建传感器对象
sensors = RobotDogSensors()

# 读取位姿
roll, pitch, yaw = sensors.read_pose()
print(f"Pose angles: Roll={roll}, \
    Pitch={pitch}, Yaw={yaw}")
```

读取表示机器狗状态的相关函数

读取机器狗姿态角度

- 在第二讲中控制机身旋转的方法：

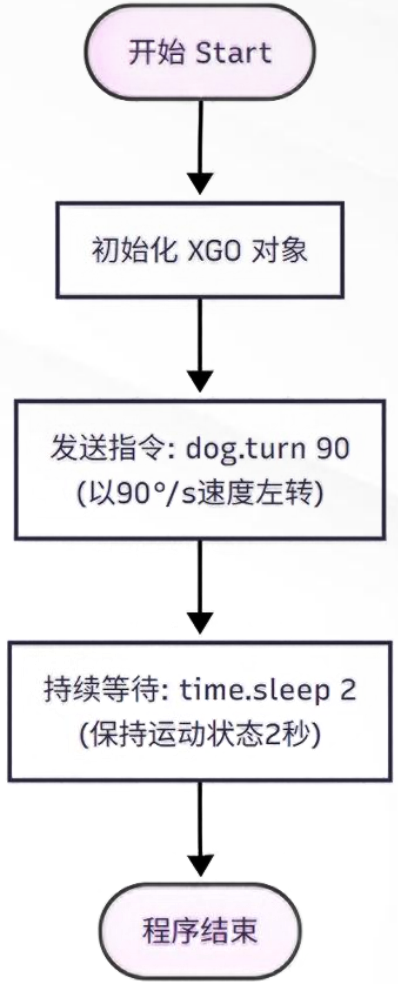
  dog.turn (speed)

  time.sleep (time)

- 直接指定了机器狗旋转的角速度speed，之后用

  角速度speed*时间time=角度angle

- 这一简单公式计算了控制机器狗旋转特定角度。
- **开环控制方式**

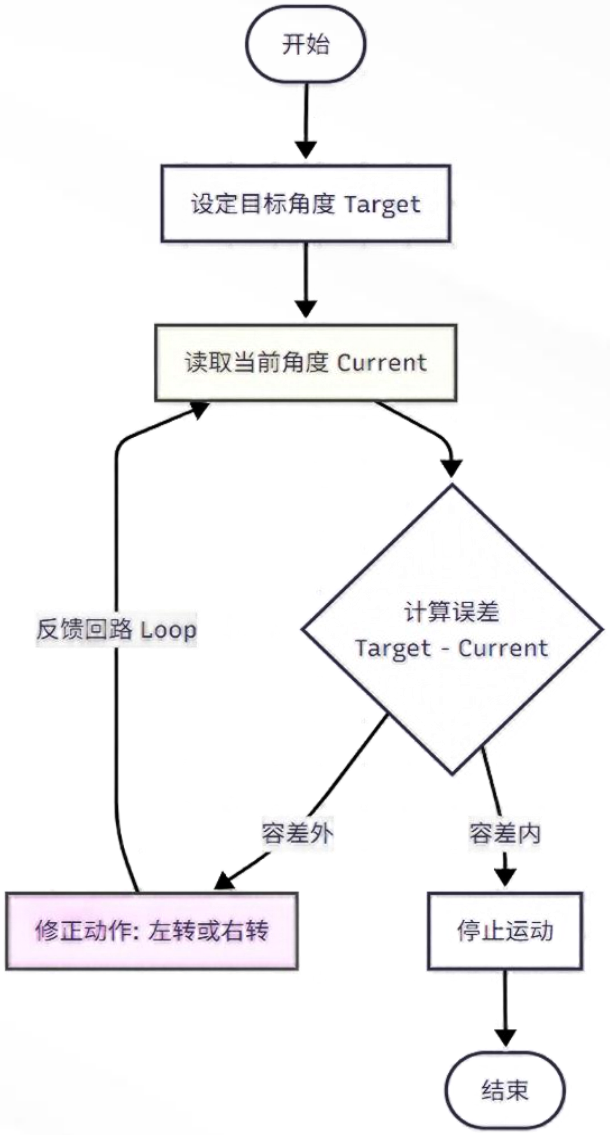


开环控制

读取相关函数

读取机器狗姿态角度

- 在本节课程中，我们引入姿态**传感器**控制角度，可以**通过传感器获取误差将角度从当前量控制到给定量。**
- 对比开环控制，其意义在于赋予了系统感知偏差并自我纠错的能力。
- 通过实时监测实际状态与期望目标的误差，动态调整控制指令来对抗环境干扰（如地面打滑）或硬件差异（如电压波动）。
- 这种负反馈机制确保了机器狗不再是盲目地“走完流程”，而是能够根据实际情况不断修正轨迹，从而保证了最终控制结果的高精度。



闭环控制

```python
from xgolib import XGO
import time

dog = XGO("xgolite")
targets = [120, 60, 0]
speed = 30
threshold = 5

for target in targets:
    print(f"目标: {target}°")
    while True:
        time.sleep(0.1)
        current = dog.read_yaw()
        if current == 0: continue
        error = target - current
        # 角度环绕处理
        if error > 180: error -= 360
        if error < -180: error += 360
        if abs(error) <= threshold:  # 到达
            dog.stop()
            time.sleep(2)
            break
        elif error > 0:  # 左转
            dog.turn(speed)
        else:  # 右转
            dog.turn(-speed)

dog.stop()
```

读取相关函数

读取舵机角度

read-motor ()

部位	1.末端	2.臂身	3.基座
1.左前腿	11	12	13
2.右前腿	21	22	24
3.右后腿	31	32	33
4.左后腿	41	42	43
5.机械臂	51（夹爪）	52（小臂）	53（大臂）

- 读取15个舵机的角度， 读取成功则返回长度为15的列表
- 对应编号[11,12,13,21,22,23,31,32,33,41,42,43,51,52,53]的舵机角度
- 读取失败则返回空列表。

设定舵机角度

- 和读取舵机角度相对应的是设置舵机角度data 来控制编号为motor_id的舵机旋转角度。

motor(motor_id, data)

参数名	格式	输入范围	说明
motor_id	整数或整数列表	[11,12,13,21,22,23,31,32,33,41,42,43,51,52,53]	第一位数字代表舵机所在的腿，第二位数字代表在该腿上的位置，从下到上依次是1，2，3 51、52、53分别是夹爪、小臂、大臂舵机 51推荐使用claw命令来控制比较直观
data	数字或数字列表	Mini 下:[-73, 57], 中:[-66, 93], 上:[-31, 31] 51：[-65, 65] 52：[-85, 50] 53：[-75, 90]	该参数代表舵机角度位置，单位为°

读取相关函数

动作复现

- 关于读取舵机角度，我们给出一个例程。该程序一次性读取并返回所有舵机角度。
- 课堂实践：以一定采样率采样10s，如1hz，记录机器狗各个舵机角度的程序，并且把角度保存起来，从而记录了一个动作组。
- 之后我们就可以通过设置舵机角度的：

motor(motor_id, data)

把这段运动重放。

```python
from xgolib import XGO

class RobotDogSensors:
    def __init__(self):
        self.dog = XGO("xgolite")

    def read_servo_angles(self):
        """
        读取并返回所有舵机的角度列表。
        """
        servo_angles = self.dog.read_motor()
        return servo_angles

# 创建传感器对象
sensors = RobotDogSensors()

# 读取舵机角度
servo_angles = sensors.read_servo_angles()
print(f"Servo angles: {servo_angles}")
```

02

教育库基础功能

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

教育库简介

目标

- 学习如何控制MAX机器人的摄像头。
- 在这一节引入一个新的python包：xgoedu 教育库。
- XGO的教育库主要是给出了集成在AI模组中的摄像头，屏幕，按键，麦克风和喇叭等硬件的Python接口，以及部分AI模型调用等功能。

```
# 创建XGO教育库的实例
XGO_edu = XGOEDU()
```

pip install XGOEDU

教育库基础功能

屏幕绘图基础

在XGO机器狗上尝试使用屏幕相关编程之前，需要先关掉系统启动时的进程，否则会产生屏幕黑屏或者花屏的错误，命令及方法如下：

sudo ps -ef | grep main.py

sudo kill xxx

```
pi@pi:~$ sudo ps -ef | grep main.py
root         956     900  0 11:05 ?        00:00:00 su - pi -c cd /home/pi/RaspberryPi-CM5 && source xgovenv/
bin/activate && exec python common/main.py >> /home/pi/app.log 2>&1
pi          1000     956  7 11:05 ?        00:00:06 python common/main.py
pi         34464   11648  0 11:06 pts/3    00:00:00 grep main.py
pi@pi:~$ sudo kill 956
pi@pi:~$ sudo ps -ef | grep main.py
pi         40034   11648  0 11:07 pts/3    00:00:00 grep main.py
pi@pi:~$
```

教育库基础功能

屏幕绘图基础

使用以下函数在屏幕上绘制图案

使用lcd_line函数在屏幕上绘制一条线：

```
# 画一条红色线条
XGO_edu.lcd_line(50, 50, 200, 50, color=(255, 0, 0))
```

使用lcd_round函数在屏幕上绘制一个圆：

```
# 画一个红色圆，圆心在(150, 150)，半径为30，宽度为5
XGO_edu.lcd_round(150,150,30,color=(255,0,0),width=5)
```

使用lcd_rectangle函数在屏幕上绘制一个矩形：

```
# 画一个红色矩形，左上角坐标(60, 60)，右下角坐标(200, 200)
XGO_edu.lcd_rectangle(60, 60, 200, 200, outline=(255, 0, 0))
```

教育库基础功能

文字与图片显示

- 使用lcd_text函数在屏幕上显示文字：

```
# 在屏幕上显示文字"Hello, XGO!"
XGO_edu.lcd_text(50, 50, 'Hello, XGO!', color=(255, 0, 0), fontsize=20)
# 中文同样可以支持
XGO_edu.lcd_text(50, 100, '你好, XGO机器狗!', color=(255, 0, 0), fontsize=20)
```

- 使用lcd_picture函数在屏幕上显示图片：
- 为方便使用，图像相关文件的路径为/home/pi/xgoPictures，可在vscode中通过ssh传输文件。

```
# 显示图片，确保图片文件名和路径正确
XGO_edu.lcd_picture('photo.jpg')
```

教育库基础功能

按键检测与屏幕清除

- 按键检测

xgoButton(button)

参数名	格式	输入范围	返回值
button	指定字符串	["a","b"]	False未按下 True按下

- 清除屏幕

lcd_clear()

代码示例：在屏幕上显示新的文字或图形，表示按键已被检测：

```
# 显示新的文字
XGO_edu.lcd_text(50, 50, 'Button A Pressed', color=(255, 0, 0), fontsize=20)
```

教育库基础功能

音视频功能

- 播放音频 xgoSpeaker ()

xgoSpeaker(filename)

参数名	格式	输入范围	说明
filename	字符串		音频文件扩展名wav，路径为/home/pi/xgoMusic

- 录制音频 xgoAudioRecord ()

xgoAudioRecord(filename="record",seconds=5)

参数名	格式	输入范围	说明
filename(可缺省) 默认"record"	字符串		录制音频的文件名 会自动添加扩展名wav，录制的文件路径为/home/pi/xgoMusic
seconds(可缺省) 默认为5	数字		录制文件的长度(秒)

教育库基础功能

音视频功能

- 播放视频 xgoVideo ()

xgoVideo(filename)

参数名	格式	输入范围	说明
filename	字符串		视频文件扩展名mp4 路径为/home/pi/xgoVideos

- 录制视频 xgoVideoRecord()

xgoVideoRecord(filename="record",seconds=5)

参数名	格式	输入范围	说明
filename(可缺省) 默认"record"	字符串		录制视频的文件名 会自动添加扩展名mp4,录制的文件路径为/home/pi/xgoVideos
seconds(可缺省) 默认为5	数字		录制文件的长度(秒)

03

OpenCV基础

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

Opencv简介

- OpenCV (Open Source Computer Vision Library)。一个开源的 计算机视觉与图像处理库
- 由 Intel 发起，支持 C++、Python、Java等多种语言
- 可以在Windows、Linux、macOS、Android、iOS等平台运行



图像处理

视频处理

特征检测与匹配

目标检测与识别

几何形状处理

图像处理-读取图片

- 读取图片：cv2.imread(img_path)
- 彩色转灰度：gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
- 保存图片：cv2.imread(save_path, gray)

```
import cv2

# 图片路径
img_path = "/home/pi/Pictures/fig1.png"
# 读取图片并做异常判断
img = cv2.imread(img_path)
if img is None:
    print(f"错误：无法读取图片，请检查路径 {img_path} 是否正确/文件是否完好")
else:
    # 彩色转灰度
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # 关键：不显示，直接保存灰度图（命名加_gray区分原文件）
    save_path = "/home/pi/Pictures/fig1_gray.png"
    cv2.imwrite(save_path, gray)
    print(f"灰度图已成功保存！路径：{save_path}")
```

图像处理-图像缩放

cv2.imread(img_path)

```
import cv2

# 1. 配置路径（根据你的实际情况修改）
# 原图路径（建议用绝对路径，避免找不到文件）
img_path = "/home/pi/Pictures/ball.jpg"
# 缩小版图片的保存路径
small_save_path = "/home/pi/Pictures/fig1_small.png"
# 200x200版图片的保存路径
resize_200x200_save_path = "/home/pi/Pictures/fig1_fix.png"
# 2. 读取图片
img = cv2.imread(img_path)
```

图像处理-图像缩放

cv2.resize(src, dsize, dst=None, fx=0, fy=0)

参数：

- src：图片numpy数组
- dst：目标图像的大小，元组(w, h)
- fx：宽度缩放因子，0.5表示原来的一半
- fy：高度缩放因子，2.0表示原来的两倍
- 使用fx，fy时，dst可设置为(0, 0)。

```
# 3. 检查图片是否读取成功
if img is None:
    print(f"错误：无法读取图片 {img_path}，请检查文件路径或文件名是否正确！")
else:
    # 4. 调整图片尺寸
    # 方式1：按比例缩小50%（保持原图宽高比）
    small = cv2.resize(img, (0, 0), fx=0.5, fy=0.5)
    # 方式2：强制缩放到200x200像素（可能拉伸/压缩）
    resize_200x200 = cv2.resize(img, (200, 200))

    # 5. 保存调整后的图片（核心修改：替换显示为保存）
    # 保存缩小版
    cv2.imwrite(small_save_path, small)
    # 保存200x200版
    cv2.imwrite(resize_200x200_save_path, resize_200x200)

    # 打印提示，确认保存成功
    print(f"图片保存完成！")
    print(f"- 缩小50%的图片：{small_save_path}")
    print(f"- 200x200尺寸的图片：{resize_200x200_save_path}")
```

图像处理-图像裁剪

- cropped_img = img[row_start:row_end, col_start:col_end]
- img[100:300, 200:400]可以看成img[row, coloumn]，也就是取图片100-300之间的行，和200-400的列。

```python
import cv2

# 1. 配置参数（改这几行就行）
input_path = "/home/pi/Pictures/fig1.png"     # 你的原图路径
output_path = "/home/pi/Pictures/fig3.png"  # 剪裁后的保存路径
row_start, row_end = 100, 300  # 行范围（高度）
col_start, col_end = 200, 400  # 列范围（宽度）

# 2. 读取图片
img = cv2.imread(input_path)

# 3. 核心：剪裁（行范围，列范围）
cropped_img = img[row_start:row_end, col_start:col_end]

# 4. 保存剪裁结果
cv2.imwrite(output_path, cropped_img)
print(f"剪裁完成！已保存至 {output_path}")
```

图像处理-图像平滑与去噪

图像平滑（模糊处理）的核心是通过算法降低图像噪声、让图像更柔和，不同模糊方法的适用场景不同。

- 高斯模糊：blurred_img = cv2.GaussianBlur(img, (5, 5), 0)

  去除高斯噪声（如摄像头噪点、光线噪点），模糊更自然，保留边缘。

- 均值模糊：blurred_img = cv2.blur(img, (5, 5))

  快速去除均匀噪声，降低图像细节，模糊较生硬，速度快。

- 中值模糊：blurred_img = cv2.medianBlur(img, 5)

  去除椒盐噪声（如黑白斑点、图像毛刺），保护边缘，适合颗粒噪点。

图像处理-图像平滑与去噪

- 示例中用5（5x5 的核）
- 核越大尺子覆盖的像素越多，参与计算的像素就越多，模糊效果越强，但图像细节丢失也越多
- **高斯 / 均值模糊**：核大小是元组（如(5,5)），可设置非正方形（如(3,7)）
- **中值模糊**：核大小是单个数字（如5），只能是奇数。

```python
import cv2
import numpy as np

img = cv2.imread("/home/pi/Pictures/crowd.jpeg")

#执行三种模糊处理（核大小建议用奇数，如3/5/7，越大模糊越强）
# 高斯模糊：参数(5,5)是核大小，0是X方向标准差（自动计算）
gaussian_blur = cv2.GaussianBlur(img, (5, 5), 0)
# 均值模糊：参数(5,5)是核大小，邻域内像素取平均
mean_blur = cv2.blur(img, (5, 5))
# 中值模糊：参数5是核大小，邻域内像素取中值（无核大小元组）
median_blur = cv2.medianBlur(img, 5)

#保存处理后的图像（可选，便于后续分析
cv2.imwrite("/home/pi/Pictures/gaussian_blur.jpg", gaussian_blur)
cv2.imwrite("/home/pi/Pictures/mean_blur.jpg", mean_blur)
cv2.imwrite("/home/pi/Pictures/median_blur.jpg", median_blur)

print("图像已保存")
```

图像处理-图像边缘检测

图像边缘检测的核心是**识别图像中像素值突变的区域（边缘）**，这些区域对应物体的轮廓、边界，是目标识别（如小球识别）、图像分割的关键步骤。

- 边缘检测算法的核心是计算像素的**梯度（变化率）**：
  - 梯度值越大 → 像素变化越剧烈 → 判定为边缘；
  - 梯度值越小 → 像素变化平缓 → 判定为非边缘。

- 进行图像边缘检测的经典方法：
  - Sobel/Laplacian 是计算梯度的算子（基础工具）
  - Canny 是完整流程的边缘检测算法

图像处理-图像边缘检测

Sobel 算子（分方向检测边缘）：

- sobel = cv2.Sobel(src, ddepth, dx, dy, ksize=None)
- Sobel 是「一阶梯度算子」，专门计算水平（X 方向）或垂直（Y 方向）的边缘，输出的是「梯度图」（非二值图），像素值代表梯度强度（边缘的明显程度）。

```
#Sobel 算子
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=5)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=5)

sobel_x_abs = cv2.convertScaleAbs(sobel_x)
sobel_y_abs = cv2.convertScaleAbs(sobel_y)
sobel_combined = cv2.addWeighted(sobel_x_abs, 0.5, sobel_y_abs, 0.5, 0)
```

图像处理-图像边缘检测

Laplacian 算子（检测所有方向边缘）：

- laplacian = cv2.Laplacian(src, ddepth, ksize=None)
- Laplacian 是「二阶梯度算子」，计算像素的二阶导数，能检测「所有方向的边缘」（无 X/Y 方向之分）
- 但对噪点极敏感，通常需先做高斯模糊（也叫「Laplacian of Gaussian，LoG」）。

```
#laplacian 算子
laplacian = cv2.Laplacian(img_blur, cv2.CV_64F, ksize=3)
laplacian_abs = cv2.convertScaleAbs(laplacian)
```

图像处理-图像边缘检测

Canny 边缘检测： edges = cv2.Canny(img, threshold1, threshold2)

- Canny 不是单一算子，而是一套完整的边缘检测流程（高斯模糊去噪→梯度计算→非极大值抑制→双阈值筛选→边缘连接）
- 最终输出干净的二值边缘图（边缘白、背景黑）
- 实际项目（如小球轮廓识别）的首选。

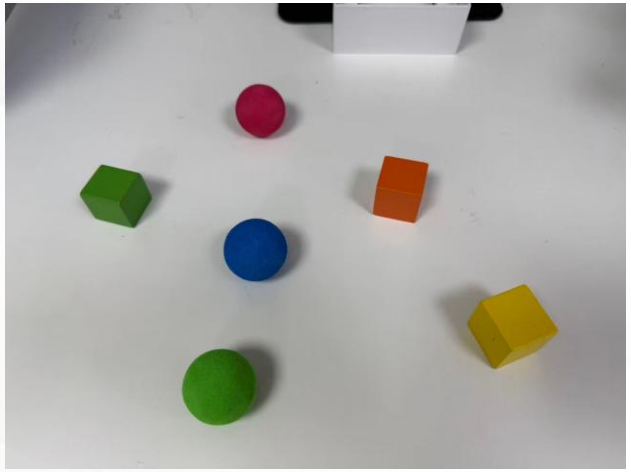
  - threshold1：梯度值低于该值 → 直接判定为非边缘，舍弃；
  - threshold2：梯度值高于该值 → 直接判定为强边缘，保留。
  - 低阈值（100）、高阈值（200）是「边缘筛选的门槛」
  - 阈值设置太高（如 200→300）：边缘会变少，可能丢失细边缘（如小球的轻微轮廓）
  - 阈值设置太低（如 100→50）：会检测出大量噪点，边缘图杂乱。

图像处理-图像边缘检测

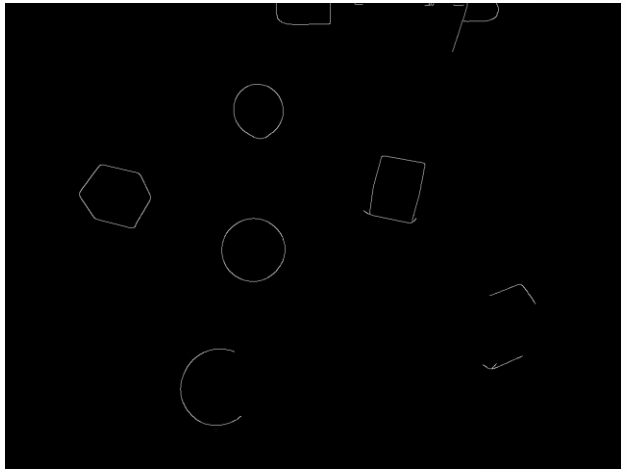
Canny 边缘检测：

```
import cv2
# 读取图像并去噪（关键：Canny前去噪能大幅提升效果）
img = cv2.imread("/home/pi/Pictures/ball.jpg", 0)  # 0表示读取为灰度图（边缘检测无需彩色）
img_blur = cv2.GaussianBlur(img, (3,3), 0)

# Canny边缘检测
edges = cv2.Canny(img_blur, 100, 200)
```

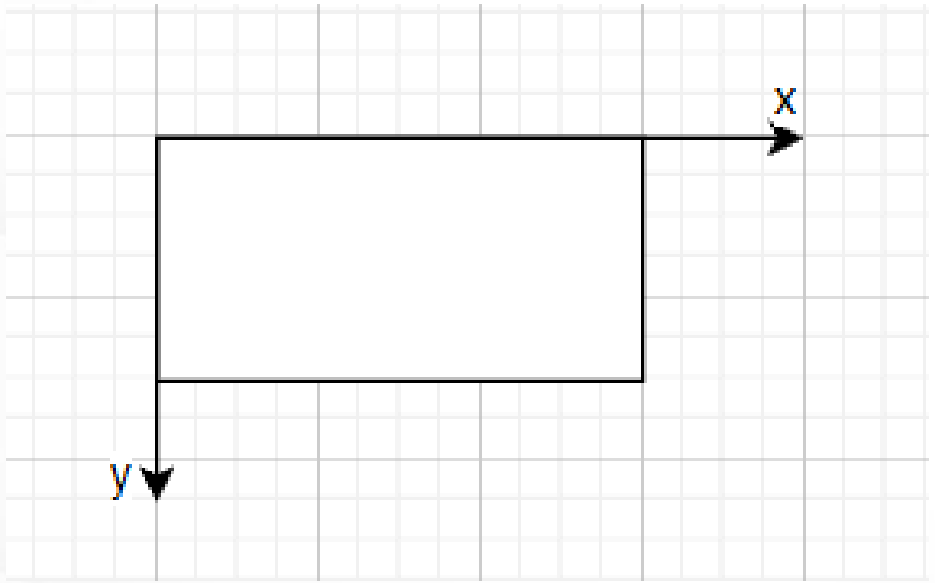


输出边缘图



绘制图形

在Opencv中，图像显示坐标：以图像左上角为原点，水平向右为x轴正方向，竖直向下为y轴正方向。



功能	函数
画线	`cv2.line()`
画矩形	`cv2.rectangle()`
画圆	`cv2.circle()`
画椭圆	`cv2.ellipse()`
画多边形	`cv2.polylines()`
文字	`cv2.putText()`

参数	含义	示例
`color`	颜色，BGR 格式	`(255,0,0)` 蓝色
`thickness`	线宽（像素）	`1`、`2`、`-1`（填充）
`lineType`	线条样式	`cv2.LINE_8`、`cv2.LINE_AA`（抗锯齿）

绘制图形


```python
# ==================== 绘制形状 ====================
# 1. 绘制绿色直线（起点→终点，线宽3）
cv2.line(img, (50, 100), (450, 100), (0, 255, 0), thickness=3)

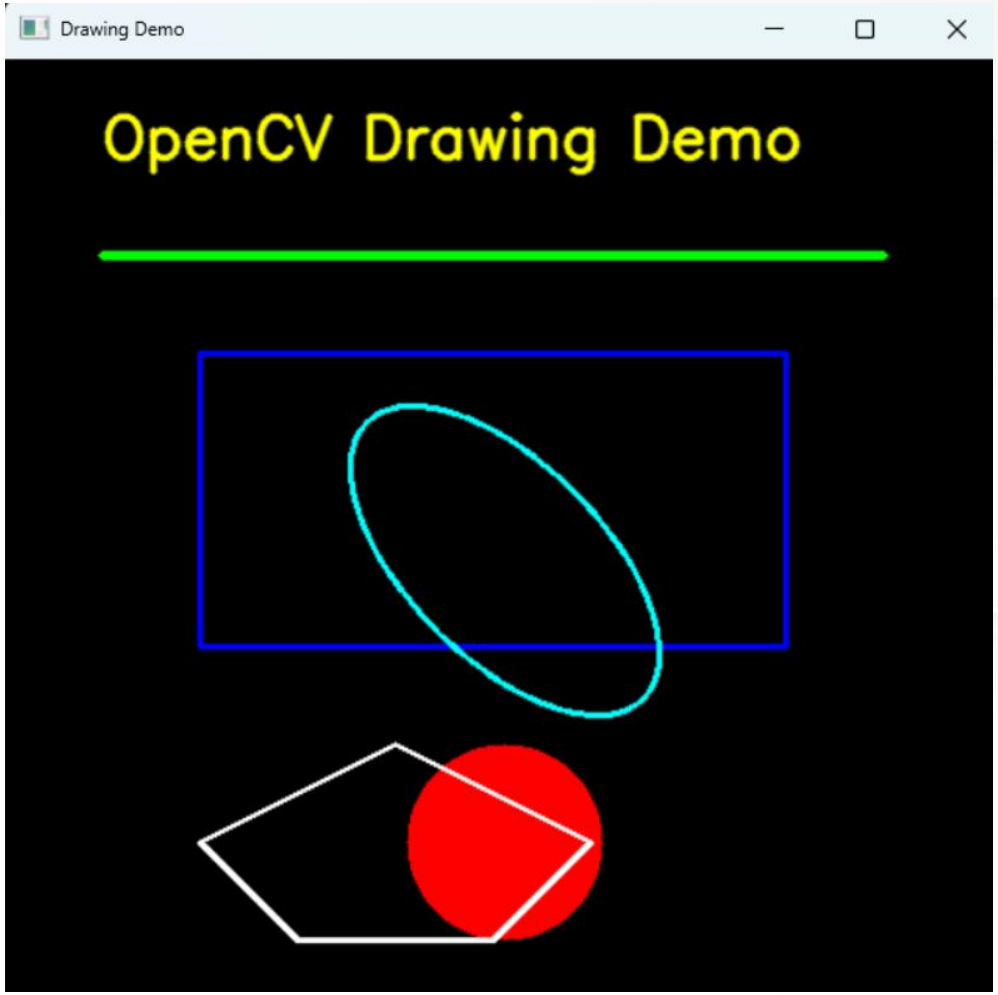
# 2. 绘制蓝色矩形（左上角→右下角，线宽2，-1为填充）
cv2.rectangle(img, (100, 150), (400, 300), (255, 0, 0), thickness=2)

# 3. 绘制红色实心圆（圆心，半径，实心）
cv2.circle(img, (256, 400), 50, (0, 0, 255), thickness=-1)

# 4. 绘制青色椭圆（中心点，长短轴，旋转角度，起始/终止角度，线宽2）
cv2.ellipse(img, (256, 256), (100, 50), 45, 0, 360, (255, 255, 0), 2)

# 5. 绘制白色多边形（顶点坐标，闭合，线宽2）
pts = np.array([[100,400], [200,350], [300,400], [250,450], [150,450]], np.int32
pts = pts.reshape((-1, 1, 2))  # 转换为OpenCV要求的形状：(点数, 1, 2)
cv2.polylines(img, [pts], isClosed=True, color=(255, 255, 255), thickness=2)

# 6. 添加黄色文字（抗锯齿，更清晰）
cv2.putText(
    img,
    "OpenCV Drawing Demo",
    (50, 50),                # 文本左下角坐标
    cv2.FONT_HERSHEY_SIMPLEX, # 字体
    1,                       # 字体大小
    (0, 255, 255),           # 颜色：黄色
    2,                       # 线宽
    cv2.LINE_AA              # 抗锯齿
)
```



形状与轮廓检测

在计算机视觉中，轮廓（Contour）是指图像中连续边缘的曲线，它能有效地描述一个物体的形状和结构特征。

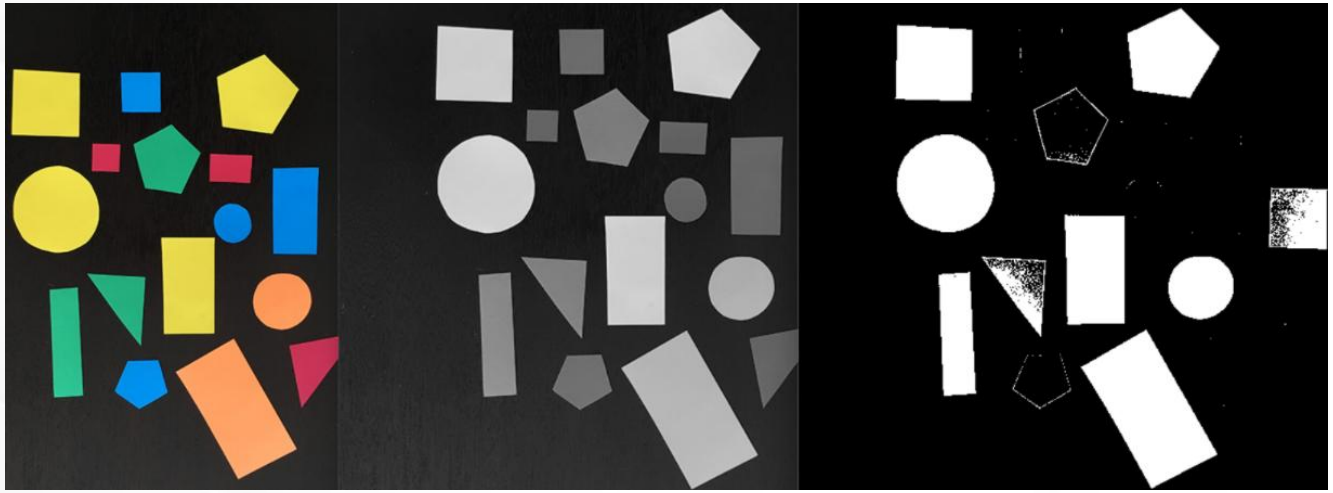
OpenCV 提供了非常强大的轮廓检测函数 `cv2.findContours()`，常用于以下任务：

- **形状识别**（判断圆形、矩形、三角形等）
- **目标分割**（检测物体区域）
- **测量与检测**（计算面积、周长、位置）
- **图像特征提取**（如边缘点、边界框）

形状与轮廓检测

- 轮廓检测的前提是：黑白分明的二值图像（即只有 0 和 255 两种像素值）。
- OpenCV 会在“颜色变化的边界”上寻找连续的像素点作为轮廓。

```
img = cv2.imread("/home/pi/Pictures/contour.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 130, 255, cv2.THRESH_BINARY)
```



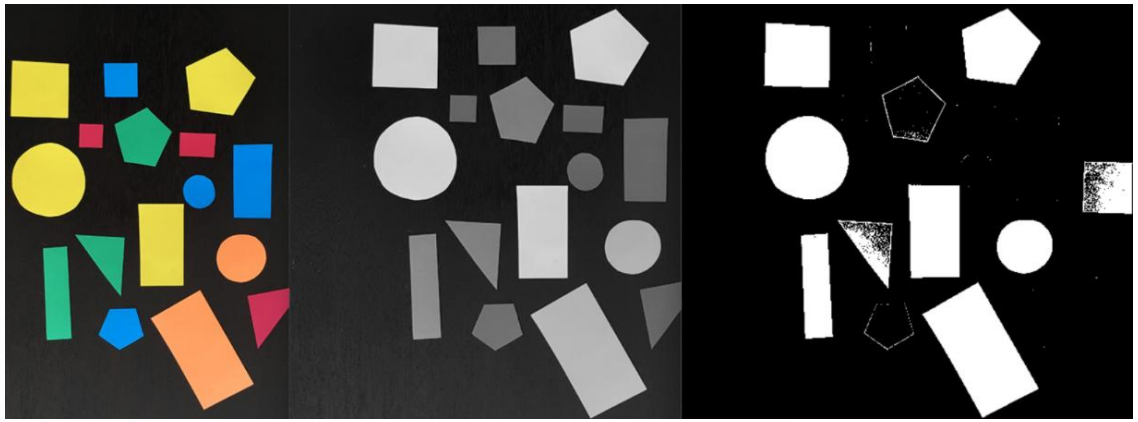
- cv2.threshold() 把灰度图转为二值图；
- 白色（255）表示目标；
- 黑色（0）表示背景。

形状与轮廓检测

cv2.findContours () 的核心任务

- 从二值图中，沿着目标的边界像素连续地走一圈，把轮廓点记录下来。
- 算法原理来自于 Suzuki & Abe 算法（1985）
- 从左上角开始扫描，遇到白点 → 判断它是否是新轮廓的起点 →若是，就开始沿边缘方向（顺时针）跟踪，直到回到起点为止。
- 最终返回所有目标的边界点序列。

```
contours, hierarchy = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```



目标识别

形状与轮廓检测

OpenCV 不直接输出“这是个矩形”或“这是个三角形”，它只输出轮廓的点的具体位置。 形状判断是通过对这些点做几何分析得到的。

多边形拟合：轮廓往往包含很多噪声点，例如矩形的边缘可能有几十个点。为了方便识别，需要“简化”轮廓。

- **cv2.arcLength(contour, True)**计算轮廓的周长；
- **epsilon**是近似精度（比例越大，形状越粗糙）；
- **cv2.approxPolyDP()** 通过Ramer–Douglas–Peucker 算法对曲线进行“折线近似”；它会用更少的点（顶点）来表示同样的形状。

```
epsilon = 0.02 * perim_px
approx = cv2.approxPolyDP(contour, epsilon, True)
```

形状与轮廓检测

顶点数判断

- 根据 len(approx)（顶点数量）来判断形状。
- 判断正方形 vs 矩形
- 对于 4 个顶点的情况，需要进一步判断长宽比例。

```
sides = len(approx)
shape = "Unknown"
if sides == 3:
    shape = "Triangle"
elif sides == 4:
    x, y, w, h = cv2.boundingRect(approx)
    aspect_ratio = float(w) / h
    shape = "Square" if 0.95 < aspect_ratio < 1.05 else "Rectangle"
elif sides > 5:
    shape = "Circle"
```

判断圆形：面积比法

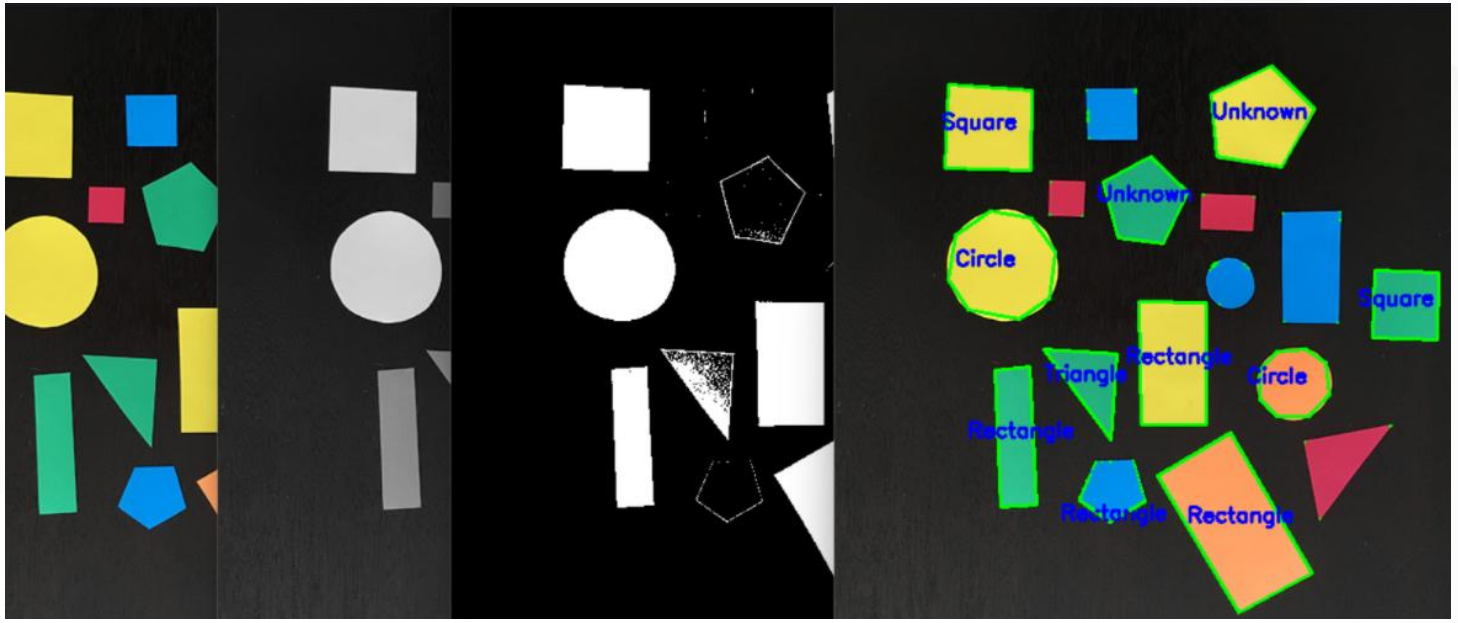
- 当轮廓点很多时（近似为圆），我们可以通过面积对比来判断。
- 如果轮廓的面积与外接圆面积非常接近，说明它很“圆滑”，即为圆形

形状与轮廓检测

cv2.moments()可以用来计算轮廓的几何矩（moments），这些矩可以用来求取形心（质心）坐标。

```
M = cv2.moments(contour)
if M["m00"] != 0:
    cx = int(M["m10"] / M["m00"])
    cy = int(M["m01"] / M["m00"])
else:
    cx, cy = 0, 0
```

形状检测结果示例



目标识别

人脸检测

- 人脸检测（Face Detection） 是 OpenCV 最常用、最经典的计算机视觉功能之一。它可以用于：摄像头实时人脸识别、拍照自动对焦、表情分析、身份验证等场景。
- OpenCV 提供了**预训练的 Haar 特征分类器**，可以直接用于人脸检测。这些分类器以 XML 文件的形式存储，包含了训练好的模型参数。
- OpenCV 中的 cv2.CascadeClassifier() 类用于加载和使用这些分类器。

```
import cv2

# 加载 Haar 特征分类器
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
```

人脸检测

人脸检测的实现步骤

- 加载 Haar 特征分类器模型: 使用 cv2.CascadeClassifier() 加载预训练的人脸检测模型
- 读取图像: 使用 cv2.imread() 读取待检测的图像。
- 转换为灰度图: 将图像转换为灰度图，因为 Haar 特征分类器在灰度图上运行更快。
- 检测人脸: 使用 detectMultiScale() 方法检测图像中的人脸。
- 绘制检测结果: 在图像中绘制检测的人脸矩形框。

```python
import cv2

# ==================== 配置项 ====================
IMAGE_PATH = "/home/pi/Pictures/OIP-C.jpg"  # 待检测的图片路径
SAVE_PATH = "/home/pi/Pictures/face_detected.jpg"  # 检测结果保存路径
# Haar分类器路径（适配OpenCV所有版本）
CASCADE_PATH = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'

def detect_face_and_save():
    # 2. 加载Haar特征分类器
    face_cascade = cv2.CascadeClassifier(CASCADE_PATH)
    if face_cascade.empty():
        print(f"❌ 错误：Haar分类器加载失败 → {CASCADE_PATH}")
        return
    print("✅ Haar人脸分类器加载成功")

    # 3. 读取图像
    image = cv2.imread(IMAGE_PATH)
    if image is None:
        print(f"❌ 错误：无法读取图片 → {IMAGE_PATH}")
        return
    print(f"✅ 成功读取图片：{IMAGE_PATH}（尺寸：{image.shape[1]}×{image.shape[0]}）")

    # 4. 转换为灰度图像
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # 5. 进行人脸检测
    faces = face_cascade.detectMultiScale(
        gray_image,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(30, 30)
    )
    print(f"🔍 检测到 {len(faces)} 个人脸")

    # 6. 绘制检测框
    for (x, y, w, h) in faces:
        cv2.rectangle(image, (x, y), (x+w, y+h), (255, 0, 0), 2)
        print(f"📌 人脸位置：左上角({x},{y})，宽{w}，高{h}")

    # 7. 保存检测结果（核心：替代imshow显示）
    cv2.imwrite(SAVE_PATH, image)
    print(f"✅ 检测结果已保存到：{SAVE_PATH}")

if __name__ == "__main__":
    detect_face_and_save()
```

目标识别—人脸检测

运行代码，可以看到图片中的人脸被绿色矩形框标出。



```python
import cv2

# ==================== 配置项 ====================
IMAGE_PATH = "/home/pi/Pictures/OIP-C.jpg"  # 待检测的图片路径
SAVE_PATH = "/home/pi/Pictures/face_detected.jpg"  # 检测结果保存路径
# Haar分类器路径（适配OpenCV所有版本）
CASCADE_PATH = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'

def detect_face_and_save():
    # 2. 加载Haar特征分类器
    face_cascade = cv2.CascadeClassifier(CASCADE_PATH)
    if face_cascade.empty():
        print(f"❌ 错误：Haar分类器加载失败 → {CASCADE_PATH}")
        return
    print("✅ Haar人脸分类器加载成功")

    # 3. 读取图像
    image = cv2.imread(IMAGE_PATH)
    if image is None:
        print(f"❌ 错误：无法读取图片 → {IMAGE_PATH}")
        return
    print(f"✅ 成功读取图片：{IMAGE_PATH}（尺寸：{image.shape[1]}×{image.shape[0]}）")

    # 4. 转换为灰度图像
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # 5. 进行人脸检测
    faces = face_cascade.detectMultiScale(
        gray_image,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(30, 30)
    )
    print(f"🔍 检测到 {len(faces)} 个人脸")

    # 6. 绘制检测框
    for (x, y, w, h) in faces:
        cv2.rectangle(image, (x, y), (x+w, y+h), (255, 0, 0), 2)
        print(f"📌 人脸位置：左上角({x},{y})，宽{w}，高{h}")

    # 7. 保存检测结果（核心：替代imshow显示）
    cv2.imwrite(SAVE_PATH, image)
    print(f"✅ 检测结果已保存到：{SAVE_PATH}")

if __name__ == "__main__":
    detect_face_and_save()
```

课堂实践

上海交通大学130周年校庆主题

机器狗

- 在机器狗屏幕上设计并绘制130周年校庆图案，甚至可以是动态的
- 播放相关主题音乐

opencv

- 校庆主题图像预处理
- 输出图像尺寸、通道数
- 将图像缩放到统一尺寸，例如800x600
- 分别生成：灰度图、二值图、边缘图
- 形状与轮廓检测、识别





谢谢！

人工智能实践教学中心
Artificial Intelligence Practice Teaching Center